

Unit -2

Introduction to numpy

- NumPy stands for Numerical Python.
- It is a Python library used for scientific computing, mathematical operations, and handling large multi-dimensional arrays and matrices.
- It also provides a wide range of mathematical, logical, statistical, and algebraic functions.
- Faster than Python lists (uses optimized C libraries).
- Provides multi-dimensional arrays (ndarray).
- Efficient element-wise operations (add, subtract, multiply, divide).
- Supports linear algebra, Fourier transforms, random numbers, etc.
- Backbone of many libraries: Pandas, TensorFlow, SciPy, Scikit-learn.

Installing NumPy

```
pip install numpy
```

Importing NumPy

```
import numpy as np
```

NumPy Arrays

Unlike Python lists, NumPy arrays are homogeneous (all elements are of the same type) and allow vectorized operations.

Creating Arrays

```
import numpy as np
```

```
# From Python list
```

```
arr = np.array([1, 2, 3, 4, 5])
```

```
print(arr)
```

```
# 2D Array
```

```
mat = np.array([[1, 2, 3], [4, 5, 6]])
```

```
print(mat)
```

Useful Array Creation Functions

```
np.zeros((3,3))      # 3x3 array of zeros
```

```
np.ones((2,4))       # 2x4 array of ones
```

```
np.arange(0,10,2)    # [0,2,4,6,8]
```

```
np.linspace(0,1,5)   # 5 values between 0 and 1
```

```
np.eye(3)            # 3x3 Identity matrix
```

Basic Array Operations

```
a = np.array([10, 20, 30, 40])
```

```
b = np.array([1, 2, 3, 4])
```

```
print(a + b)  # [11 22 33 44]
```

```
print(a - b)  # [ 9 18 27 36]
```

```
print(a * b)  # [10 40 90 160]
```

```
print(a / b)  # [10. 10. 10. 10.]
```

Array Properties

```
arr = np.array([[1,2,3],[4,5,6]])  
  
print(arr.shape)    # (2, 3)  
  
print(arr.ndim)     # 2 (2D array)  
  
print(arr.size)     # 6  
  
print(arr.dtype)    # int64 (depends on system)
```

Indexing & Slicing

```
arr = np.array([10,20,30,40,50])  
  
print(arr[0])       # 10  
  
print(arr[-1])      # 50  
  
print(arr[1:4])     # [20 30 40]
```

```
mat = np.array([[1,2,3],[4,5,6],[7,8,9]])  
  
print(mat[0,2])     # 3  
  
print(mat[:,1])     # [2 5 8] (column)  
  
print(mat[1,:])     # [4 5 6] (row)
```

Mathematical Functions

```
arr = np.array([1,2,3,4,5])  
  
print(np.mean(arr)) # 3.0  
  
print(np.median(arr)) # 3.0  
  
print(np.std(arr))  # 1.41...  
  
print(np.sum(arr))  # 15
```

```
print(np.max(arr))    # 5
```

Creation of vectors and metrics

Vectors and Matrices in NumPy

1. Creating Vectors

A vector is simply a 1-D NumPy array.

a) From a Python List

```
import numpy as np
```

```
# Row vector
```

```
v1 = np.array([1, 2, 3, 4, 5])
```

```
print("Vector v1:", v1)
```

b) Using `arange()`

```
v2 = np.arange(1, 11, 2)    # start=1, stop=11, step=2
```

```
print("Vector v2:", v2)    # [ 1  3  5  7  9]
```

c) Using `linspace()`

```
v3 = np.linspace(0, 1, 5)  # 5 values equally spaced between 0 and 1
```

```
print("Vector v3:", v3)    # [0.   0.25 0.5   0.75 1. ]
```

2. Creating Matrices

A matrix is a 2-D NumPy array.

a) From Nested Lists

```
M1 = np.array([[1, 2, 3],
```

```
               [4, 5, 6],
```

```
               [7, 8, 9]])
```

```
print("Matrix M1:\n", M1)
```

b) Using zeros() and ones()

```
M2 = np.zeros((3, 3)) # 3x3 matrix of zeros
```

```
M3 = np.ones((2, 4)) # 2x4 matrix of ones
```

```
print("M2:\n", M2)
```

```
print("M3:\n", M3)
```

c) Using eye() (Identity Matrix)

```
I = np.eye(4) # 4x4 identity matrix
```

```
print("Identity Matrix:\n", I)
```

d) Random Matrices

```
M4 = np.random.rand(3, 3) # 3x3 matrix with values in [0,1)
```

```
M5 = np.random.randint(1, 10, (3, 3)) # 3x3 matrix with random integers 1–9
```

```
print("Random Float Matrix:\n", M4)
```

```
print("Random Integer Matrix:\n", M5)
```

3. Shape of Vectors and Matrices

```
print(v1.shape) # (5,) → vector with 5 elements
```

```
print(M1.shape) # (3, 3) → 3x3 matrix
```

Matrix Manipulation, Indexing & Slicing

Indexing Elements

NumPy uses zero-based indexing.

Syntax: `M[row, column]`

```
print(M[0, 0]) # First row, first column → 1
```

```
print(M[1, 2]) # Second row, third column → 6
```

```
print(M[-1, -1]) # Last row, last column → 9
```

3. Slicing Rows & Columns

```
print(M[0, :]) # First row → [1 2 3]
```

```
print(M[:, 1]) # Second column → [2 5 8]
```

```
print(M[1:3, :]) # Rows 2 to 3 → [[4 5 6], [7 8 9]]
```

```
print(M[:, ::2]) # Every 2nd column → [[1 3], [4 6], [7 9]]
```

4. Submatrices

```
print(M[0:2, 0:2]) # Top-left 2x2 → [[1 2], [4 5]]
```

```
print(M[1:, 1:]) # Bottom-right 2x2 → [[5 6], [8 9]]
```

5. Matrix Transpose

```
print(M.T) # Transpose
```

6. Reshaping & Flattening

```
# Reshape 3x3 → 1x9
```

```
print(M.reshape(1, 9))
```

```
# Flatten into 1D array
```

```
print(M.flatten())
```

7. Stacking Matrices

```
A = np.array([[1, 2], [3, 4]])
```

```
B = np.array([[5, 6], [7, 8]])
```

```
print(np.hstack((A, B))) # Horizontal stack
```

```
print(np.vstack((A, B))) # Vertical stack
```

8. Deleting Rows & Columns

```
print(np.delete(M, 0, axis=0)) # Delete first row  
print(np.delete(M, 1, axis=1)) # Delete second column
```

9. Adding Rows & Columns

```
row = np.array([[10, 11, 12]])  
col = np.array([[10], [11], [12]])  
print(np.vstack((M, row))) # Add new row  
print(np.hstack((M, col))) # Add new column
```

10. Select Multiple Rows/Cols

```
print(M[[0, 2], :]) # Select rows 1 & 3  
print(M[:, [0, 2]]) # Select cols 1 & 3
```

Introduction to Pandas in Python

1. What is Pandas?

- Pandas = Python Data Analysis Library.
- It provides fast, flexible, and easy-to-use data structures for handling structured data.
- Mainly used for data analysis, data cleaning, data manipulation, and data visualization.
- Built on NumPy and often used with Matplotlib, Seaborn, and Scikit-learn.

2. Why use Pandas?

- Easy handling of tabular data (rows & columns like Excel/SQL).
- Built-in functions for filtering, grouping, merging, reshaping.
- Works with different file formats: CSV, Excel, SQL, JSON, etc.

Provides two main data structures:

1. Series → 1D labeled array (like a column).
2. DataFrame → 2D labeled data (like a table).

Installing Pandas

```
pip install pandas
```

4. Importing Pandas

```
import pandas as pd
```

5. Pandas Data Structures

a) Series (1D Data)

```
import pandas as pd
```

```
data = [10, 20, 30, 40]
```

```
s = pd.Series(data)
```

```
print(s)
```

Output

```
0    10
```

```
1    20
```

```
2    30
```

```
3    40
```

```
dtype: int64
```

b) DataFrame (2D Data)

```
data = {'Name': ['Alice', 'Bob', 'Charlie'],
```

```
'Age': [25, 30, 35],  
'City': ['Delhi', 'Mumbai', 'Hyderabad']}]}
```

```
df = pd.DataFrame(data)  
print(df)
```

Output

	Name	Age	City
0	Alice	25	Delhi
1	Bob	30	Mumbai
2	Charlie	35	Hyderabad

6. Reading & Writing Data

Reading files

```
df = pd.read_csv("data.csv")    # from CSV  
df = pd.read_excel("data.xlsx") # from Excel  
df = pd.read_json("data.json")  # from JSON
```

Writing files

```
df.to_csv("output.csv", index=False)  
df.to_excel("output.xlsx", index=False)
```

7. Basic DataFrame Operations

```
print(df.head())    # First 5 rows
```

```
print(df.tail())      # Last 5 rows
print(df.info())      # Summary
print(df.describe())  # Statistics
print(df.columns)     # Column names
print(df.shape)       # Rows & Columns
```

8. Selecting Data

```
print(df['Name'])      # Single column
print(df[['Name','City']])# Multiple columns
print(df.iloc[0])      # First row (by index)
print(df.loc[1, 'City']) # Value at row 1, col 'City'
```

Adding New Data in a DataFrame

We can add data in two ways:

Add new columns

Add new rows

1. Adding a New Column

a) Direct Assignment

```
import pandas as pd
```

```
data = {'Name': ['Alice', 'Bob', 'Charlie'],
        'Age': [25, 30, 35]}
```

```
df = pd.DataFrame(data)
```

```
# Add new column
```

```
df['City'] = ['Delhi', 'Mumbai', 'Hyderabad']
```

```
print(df)
```

b) Add Column with Same Value

```
df['Country'] = 'India'
```

```
print(df)
```

c) Column Based on Existing Columns

```
df['Age+5'] = df['Age'] + 5
```

```
print(df)
```

2. Adding a New Row

a) Using loc[]

```
df.loc[3] = ['David', 40, 'Chennai', 'India', 45]
```

```
print(df)
```

b) Using append() (⚠️ Deprecated after Pandas 1.4, use concat instead)

```
new_row = pd.DataFrame({'Name': ['Eva'], 'Age': [28],
```

```
                        'City': ['Kolkata'], 'Country': ['India'], 'Age+5': [33]})
```

```
df = pd.concat([df, new_row], ignore_index=True)
```

```
print(df)
```

3. Inserting at Specific Column Position

```
df.insert(1, 'ID', [101, 102, 103, 104, 105])
```

```
print(df)
```

4. Updating with Conditional Values

```
df.loc[df['Age'] > 30, 'Status'] = 'Senior'
```

```
df.loc[df['Age'] <= 30, 'Status'] = 'Junior'
```

```
print(df)
```

Deleting Data from a DataFrame

You can delete rows or columns using `drop()`, `del`, or by filtering.

1. Deleting a Column

a) Using `drop()`

```
import pandas as pd
```

```
data = {'Name': ['Alice', 'Bob', 'Charlie'],  
        'Age': [25, 30, 35],  
        'City': ['Delhi', 'Mumbai', 'Hyderabad']}
```

```
df = pd.DataFrame(data)
```

```
# Drop column
```

```
df = df.drop('City', axis=1)  # axis=1 → column
```

```
print(df)
```

b) Using `del`

```
del df['Age']
```

```
print(df)
```

c) Using `pop()`

```
city = df.pop('City')  # removes and returns the column
```

2. Deleting a Row

a) By Index

```
df = df.drop(1, axis=0)    # drop row with index 1 (Bob)
print(df)
```

b) Multiple Rows

```
df = df.drop([0, 2], axis=0)    # drop rows with index 0 and 2
print(df)
```

3. Deleting with Condition

```
# Drop rows where Age < 30
df = df[df['Age'] >= 30]
print(df)
```

4. Reset Index after Deletion

```
df = df.reset_index(drop=True)
print(df)
```